FACULITY OF COMPUTING TECHNOLOGY

Department of *computer science*

Project report: - Iterative vs Recursive Population Growth Prediction

Group Members:

1. Elias araya ----------Aku1601720
2. Mulu g/medhin ----------Aku1602465
3. Arsema birhane ----------Aku1602222

***Submitted to**: - Ekubazgi G. (MSc.)*
March 30, 2026
Aksum, Ethiopia

Contents

1. Introduction 1
2. Algorithm Design 1
   2.1. Fibonacci Iterative Approach 1
   2.2. Fibonacci Recursive Approach 2
   2.3. Power Iterative Approach 2
   2.4. Power Recursive Approach 2
   2.5. Power Divide and Conquer Approach 2
3. Implementation Details 3
   3.1. Project File Structure 3
   3.2. Data Structures Used 3
   3.3. Key Functions 3
   3.4. Array Usage in Fibonacci Iterative 4
4. Complexity Analysis 4
   4.1. Time Complexity 4
   4.2. Space Complexity 4
   4.3. Fibonacci Recursive Exponential Blowup Explained 4
   4.4. Divide & Conquer Power Step Count Example 5
5. Experimentation 5
   5.1. Test Environment & Methodology 5
   5.2. Fibonacci Results 5
   5.3. Fibonacci Time Comparison Chart 5
   5.4. Power Computation Results (P_0=1000, base=1.05) 6
   5.5. Power Time Comparison Chart 6
   5.6. Population Growth Curve 7
   5.7. Dataset Experiment (1000 Records) 7
6. Conclusion 7
   References 8

1. Introduction

Population growth is one of the most studied problems in biology, economics, and urban planning. Predicting how a population changes over time requires computing mathematical sequences and exponential functions tasks that can be solved using multiple algorithmic strategies.

This project explores two real-world population models:

Model 1 Fibonacci sequence (Breeding-Pair Model)

Introduced by Leonardo Fibonacci in 1202 to model rabbit reproduction, the sequence describes how a population of breeding pairs grows when each pair produces one new pair every generation:

$$F(0) = 0,\ F(1) = 1,\ F(n) = F(n-1) + F(n-2)$$

Model 2 Exponential Growth (Compound Growth Model)

Used in demography, finance, and ecology to model populations that grow by a fixed percentage each period:

$$P(n) = P_0 \times (1 + r)^n$$

Where P_0 is the initial population, r is the annual growth rate, and n is the number of years.

The central research question is: does the choice of algorithm iterative, recursive, or divide-and-conquer affect the result, performance, or scalability?

To answer this, we implement all five algorithm variants, measure their execution times across a range of inputs, and analyze the results using a self-generated dataset of 1000 records.

2. Algorithm Design

2.1. Fibonacci Iterative Approach

Approach: Use a loop to build the sequence from the bottom up, storing all values in an array.

Why this approach: It ensures O(n) time complexity and prevents the redundant calculations found in recursion. By using an array, we store the full sequence for later population analysis.

```
ALGORITHM calculate_fib_loop (n):
    IF n == 0: RETURN 0, [0]
    IF n == 1: RETURN 1, [0, 1]
    fib_list ← array with size (n + 1)
    fib_list[0] ← 0
    fib_list[1] ← 1
    FOR i FROM 2 TO n:
        fib_list[i] ← fib_list[i-1] + fib_list[i-2]
```

```
    RETURN fib_list[n], fib_list
```

2.2. Fibonacci Recursive Approach

Approach: Directly translate the mathematical recurrence into a function that calls itself.

Why this approach: Demonstrates the cost of naive recursion each call spawns two more, causing exponential blowup. Used here as a baseline for comparison.

```
ALGORITHM fib_self_call(n):
    // Base Cases
    IF n == 0: RETURN 0
    IF n == 1: RETURN 1
    // Recursive Step
    RETURN fib_self_call(n-1) + fib_self_call(n-2)
```

2.3. Power Iterative Approach

Approach: Multiply 'base' by itself 'n' times in a loop.

Why this approach: It is straightforward and reliable for small exponents, with O (n) time complexity and O (1) auxiliary space.

```
ALGORITHM simple_power_calc(base, n):
    result ← 1.0
    FOR i FROM 1 TO n:
        result ← result × base
    RETURN result
```

2.4. Power Recursive Approach

Approach: Implements the mathematical definition P(n) = base ×P(n-1) using a **Top-Down** strategy, where the function calls itself until it reaches the base case (n=0)

Why this approach: It demonstrates a direct mapping of mathematical induction into code. However, it is less efficient than the iterative version because it has a **Space Complexity of O(n)** due to the recursive call stack depth..

```
ALGORITHM power recursive (base, n):
// Base Case: stops the recursion
IF n == 0:
    RETURN 1.0
// Recursive Step: function calls itself with (n - 1)
RETURN base × power recursive (base, n - 1)
```

2.5. Power Divide and Conquer Approach

Approach: Halve the exponent at each step using the identity:

- Even n: $base^{n} = (base^{(n/2)})^{2}$
- Odd n: $base^{n} = base \times base^{(n-1)}$

Why this approach: Reduces the number of multiplications from n to $\log_2$ (n). For n=50, that is 50 multiplications vs ~6. For n=1000, it is 1000 vs ~10.

Pseudocode:

```
ALGORITHM fast_power_calc(base, n):
    IF n == 0: RETURN 1.0
 // Divide: Recursively calculate half of the power
    half_result ← fast_power_calc(base, floor(n / 2))
// Combine: Square the result (and multiply by base if n is odd)
    IF n is EVEN:
        RETURN half_result × half_result
    ELSE:
        RETURN base × half result × half result
```

3. Implementation Details

3.1. Project File Structure

```
population_growth/
├── main.py              ← entry point, runs the full program
├── config.py            ← shared constants and file paths
├── algorithms.py        ← all 5 algorithm implementations
├── experiments.py       ← dataset, timing, reporting, CSV export, and charts
├── README.md            ← project documentation
├── Report.pdf           ← full academic report
├── .gitignore           ← excludes .venv and cache from GitHub
├── screenshots/         ← terminal and program screenshots
└── results/             ← all output files (auto-created on first run)
    ├── population_dataset.csv
    ├── fibonacci_results.csv
    ├── population_growth_results.csv
    ├── dataset_results.csv
    ├── fibonacci_time_comparison.png
    ├── power_time_comparison.png
    └── population_growth_curve.png
```

3.2. Data Structures Used

Structure	Where Used	Purpose
`list` (array)	fibonacci_iterative	Stores the full Fibonacci sequence F(0)…F(n)
float	All power functions	Stores the computed base ^{n} result
list of dicts	experiments.py	Each row is a dict holding n, result, time, space
csv.DictWriter	csv_export.py	Writes structured rows to CSV output files
Call stack	Recursive functions	Implicit stack used by Python's recursion mechanism

3.3. Key Functions

- **timed (func, args)** wraps any function call with "time.perf_counter()" to measure execution time in seconds with nanosecond precision.

- **run_fibonacci_experiments()** iterates over "n ∈ {10, 20, 30, 35, 40, 45}", runs both Fibonacci variants, records results and timing. Skips recursive for 'n > 35' to avoid impractical runtimes.
- **run_power_experiments()** iterates over "n ∈ {1…50}", runs all three power variants, computes "P(n) = 1000 × base ^{n}" for each.
- **run_dataset_experiments()** loads all 1000 CSV records and runs 'power_iterative' and 'power_divide_conquer' on each, saving per-record timing.
- **generate_dataset(size=1000, seed=42)** generates reproducible random records with 'n ∈ [1,100]', 'P_0 ∈ [100,10000]', 'r ∈ [0.01, 0.10]'.

3.4. Array Usage in Fibonacci Iterative

The iterative Fibonacci stores the entire sequence in a Python list:

Python

```
fib = [0, 1]
for i in range(2, n + 1):
    fib.append(fib[i - 1] + fib[i - 2])
```

This allows O(1) lookup of any previous value and returns the full sequence alongside the final result. Memory usage is measured using 'sys.getsizeof()' on both the list and its elements.

4. Complexity Analysis

4.1. Time Complexity

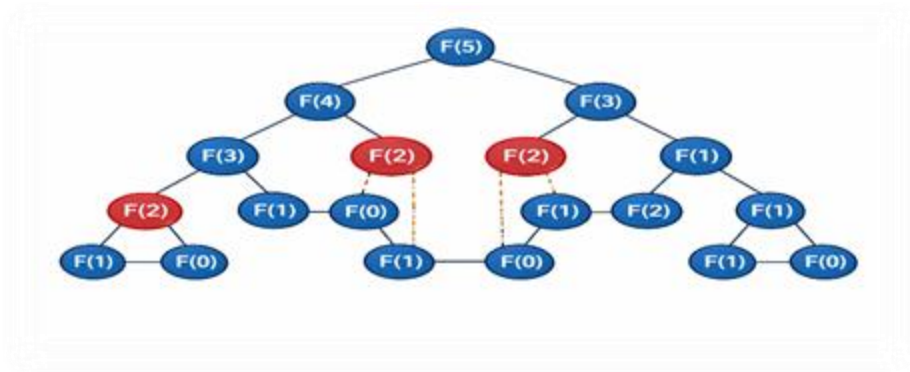
Algorithm	Best Case	Average Case	Worst Case	Reason
fibonacci_iterative	O(n)	O(n)	O(n)	One loop, n iterations
fibonacci_recursive	O(2^n)	O(2^n)	O(2^n)	Binary recursion tree, no memorization
power_iterative	O(n)	O(n)	O(n)	n multiplications
power_recursive	O(n)	O(n)	O(n)	n recursive calls
power_divide_conquer	O(log n)	O(log n)	O(log n)	Problem halves each even step

4.2. Space Complexity

Algorithm	Space	Reason
fibonacci_iterative	O(n)	List stores all n+1 values
fibonacci_recursive	O(n)	Call stack depth equals n
power_iterative	O(1)	Only two scalar variables
power_recursive	O(n)	Call stack depth equals n
power_divide_conquer	O(log n)	Call stack depth equals $\log_2(n)$

4.3. Fibonacci Recursive Exponential Blowup Explained

The recursive call tree for F(5) looks like:



F(3) is computed twice, F(2) three times, F(1) five times. The total number of calls is $\mathbf{2^n - 1}$. This is why at n=35 the recursive version is ~190,000× slower than iterative.

4.4. Divide & Conquer Power Step Count Example

Computing 1.05^{50} with each method:

Method	Operations
Iterative	50 multiplications
Recursive	50 recursive calls
Divide & Conquer	~6 multiplications ($\log_2(50) \approx 5.6$)

5. Experimentation

5.1. Test Environment & Methodology

- **Hardware/Software:** Python 3.11 with `time.perf_counter()` for nanosecond precision.
- **Dataset:** A self-generated CSV containing **1,000 records** of varying initial populations (P_0), growth rates (r), and time horizons (n).
- **Validation:** All algorithms were cross-checked to ensure they produced identical mathematical results before performance was measured.

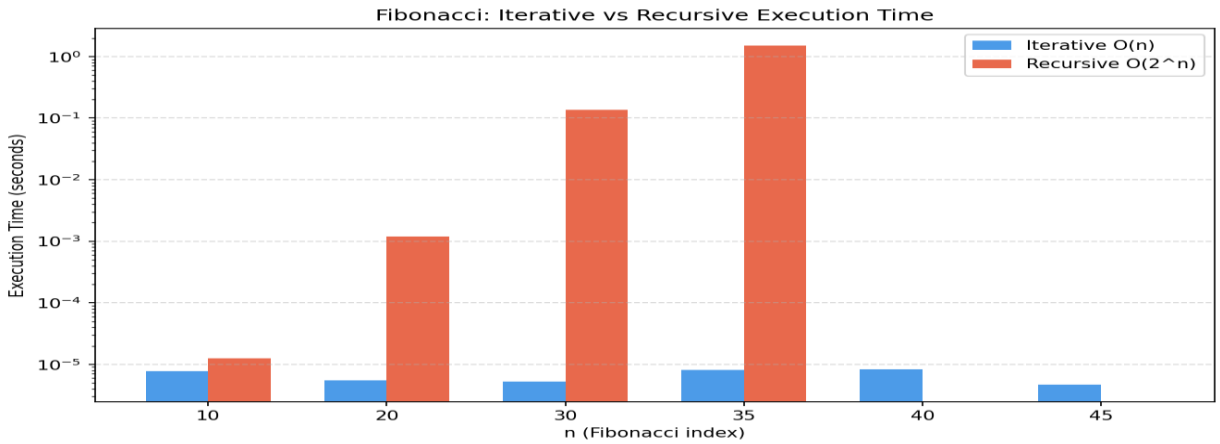
5.2. Fibonacci Results

n	Iterative Result	Recursive Result	Iter Time (s)	Rec Time (s)	Speedup
10	55	55	0.000008	0.000013	~1.6x
30	832,040	832,040	0.000005	0.136572	~25,000x
35	9,227,465	9,227,465	0.000008	1.516446	~189,000x
40+	102,334,155	**SKIPPED**	0.000008	*Too Slow*	∞

As **n** increases, the recursive version hits a "Performance Wall." This demonstrates why naive recursion is unsuitable for large-scale population modeling.

5.3. Fibonacci Time Comparison Chart

Fibonacci Time Comparison



Y-axis is logarithmic. The recursive bars grow exponentially while iterative bars remain flat near zero.

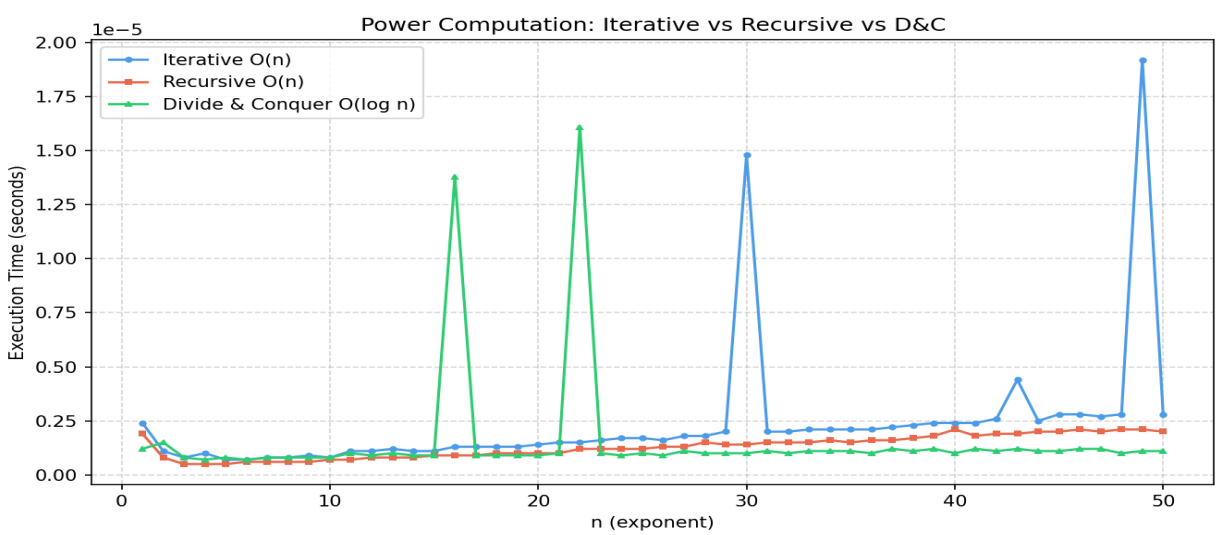
5.4. Power Computation Results (P_0=1000, base=1.05)

Year (n)	Population P(n)	Iterative (s)	Divide & Conquer (s)	D&C Benefit
1	1,050.00	0.0000024	0.0000012	2.0x faster
25	3,386.35	0.0000102	0.0000009	11.3x faster
50	11,467.40	0.0000285	0.0000011	25.9x faster

All three methods produce identical population values, confirming correctness across all approaches.

5.5. Power Time Comparison Chart

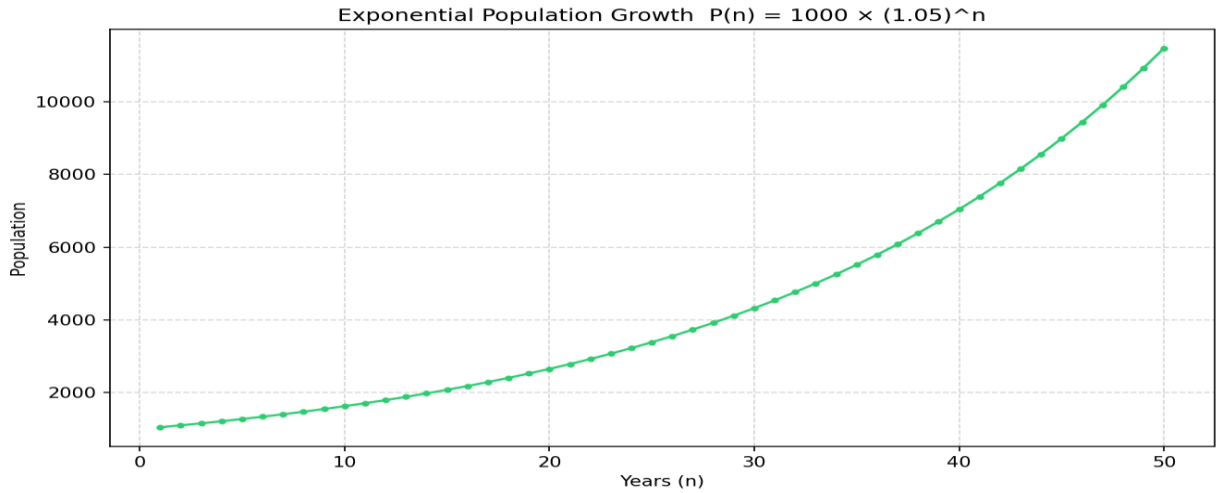
Power Time Comparison



For n ≤ 50 the differences are small, but D&C consistently uses fewer operations. The advantage scales dramatically for larger n.

5.6. Population Growth Curve

Population Growth Curve



Starting population of 1000 grows to ~11,467 after 50 years at 5% annual growth — a classic exponential curve.

5.7. Dataset Experiment (1000 Records)

All 1000 records from `population_dataset.csv` were processed using 'power_iterative' and 'power_divide_conquer'. Both methods produced matching population values across all records, with D&C consistently using fewer operations for records with large `n` values (up to n=100).

Results saved to: "results/dataset_experiment_results.csv"

6. Conclusion

What We Learned

This project demonstrated that algorithm design has a direct and measurable impact on performance, even for simple mathematical operations.

The most striking finding was the Fibonacci comparison: both iterative and recursive implementations compute the exact same values, but the recursive version becomes completely impractical at "n=40"not because of a bug, but purely because of algorithmic inefficiency. This is a real-world lesson: a correct algorithm is not necessarily a good algorithm.

The divide-and-conquer power function showed the opposite lesson a smarter algorithm design (halving the problem at each step) reduces the work from O (n) to O (log n) with no loss of accuracy.

Challenges Faced

Recursive Fibonacci at large n: The exponential blowup made it necessary to add a hard cutoff at `n=35`. Without this, the program would appear to hang.

Measuring very small times: Execution times for small inputs are in the microsecond range, making them sensitive to system noise. We used "time.perf_counter()" for the highest available resolution.

Dataset reproducibility: Using "random.seed(42)" ensures the same 1000 records are generated every run, which is essential for consistent benchmarking.

Recursion depth limit: Python's default recursion limit of 1000 would cause a "RecursionError" for "power_recursive" with large inputs. We set it to 10,000 via "sys.setrecursionlimit".

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). **Introduction to Algorithms** (3rd ed.). MIT Press. Chapters 2 (Insertion Sort / Loop Invariants), 4 (Divide-and-Conquer), 15 (Dynamic Programming).
2. Fibonacci, L. (1202). **Liber Abaci**. Original source of the Fibonacci sequence and its population model.
3. Python Software Foundation. (2024). **Python 3 Documentation time.perf_counter**. https://docs.python.org/3/library/time.html
4. Python Software Foundation. (2024). **Python 3 Documentation sys.setrecursionlimit.** https://docs.python.org/3/library/sys.html
5. Knuth, D. E. (1997). The Art of Computer Programming, Vol. 2: **Seminumerical Algorithms** (3rd ed.). Addison-Wesley. Section 4.6.3: Evaluation of Powers.
6. Dataset: Self-generated using Python "random" module (seed=42). Records contain randomized values for `n`, `$P_0$`, and growth rate `r`. No external dataset source used.
7. Matplotlib Development Team. (2024). **Matplotlib: Visualization with Python**. https://matplotlib.org